Up until now I've limited myself to describing the member function forms of the operations on the atomic types. However, there are also equivalent nonmember functions for all the operations on the various atomic types. For the most part the non-member functions are named after the corresponding member functions but with an `atomic_` prefix (for example, `std::atomic_load()`). These functions are then overloaded for each of the atomic types. Where there's opportunity for specifying a memory-ordering tag, they come in two varieties: one without the tag and one with an `_explicit` suffix and an additional parameter or parameters for the memory-ordering tag or tags (for example, `std::atomic_store(&atomic_var,new_value)` versus `std::atomic_store_explicit(&atomic_var,new_value,std::memory_order_release)`. Whereas the atomic object being referenced by the member functions is implicit, all the free functions take a pointer to the atomic object as the first parameter.

Table 5.3. The operations available on atomic types

Operation	atomic_flag	atomic<bool>	atomic<T*>	atomic<integral-type>	atomic<other-type>
test_and_set	✓				
clear	✓				
is_lock_free		✓	✓	✓	✓
load		✓	✓	✓	✓
store		✓	✓	✓	✓
exchange		✓	✓	✓	✓
compare_exchange_weak,compare_exchange_strong		✓	✓	✓	✓
fetch_add, +=			✓	✓	
fetch_sub, -=			✓	✓	
fetch_or, |=				✓	
fetch_and, &=				✓	
fetch_xor, ^=				✓	
++, --			✓	✓	

For example, `std::atomic_is_lock_free()` comes in just one variety (though overloaded for each type), and `std::atomic_is_lock_free(&a)` returns the same value as `a.is_lock_free()` for an object of atomic type `a`. Likewise, `std::atomic_load(&a)` is the same as `a.load()`, but the equivalent `of a.load(std::memory_order_acquire)` is `std::atomic_load_explicit(&a, std::memory_order_acquire)`.

The free functions are designed to be C-compatible, so they use pointers rather than references in all cases. For example, the first parameter of the `compare_exchange_weak()` and `compare_exchange_strong()` member functions (the expected value) is a reference, whereas the second parameter of `std::atomic_compare_exchange_weak()` (the first is the object pointer) is a pointer.
`std::atomic_compare_exchange_weak_explicit()` also requires both the success and failure memory orders to be specified, whereas the compare/exchange member functions have both a single memory order form (with a default of `std::memory_order_seq_cst`) and an overload that takes the success and failure memory orders separately.